Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S10: Hands-On Lab Walkthrough & CLI Guide

Spark Structured Streaming, Event-Time & Watermarking

1. Lab Objectives

In this hands-on engineering lab, you will construct a resilient real-time streaming pipeline:
1. Ingest real-time retail transaction events directly from Kafka topic `orders.events.v1` into PySpark Structured Streaming.
2. Enforce strict JSON schema parsing and extract the event-time timestamp (`timestamp`).
3. Apply an event-time Watermark (`withWatermark("timestamp", "10 minutes")`) and compute 1-hour tumbling window revenue aggregations per store.
4. Manage persistent checkpoints (`checkpointLocation`) to guarantee fault tolerance and exactly-once processing semantics.
5. Benchmark streaming micro-batch latency and throughput metrics.

2. Environment Setup & Component Verification

Verify that your virtual environment and Docker Kafka cluster are healthy:

```
cd "${LAB_ROOT:-delivery_materials/00_Lab}" 2>/dev/null || cd 00_Lab || cd ../../00_Lab

source .venv/bin/activate
if [ -d "${HOME}/.local/share/jvm/jdk-17.0.21+9" ]; then
  export JAVA_HOME="${HOME}/.local/share/jvm/jdk-17.0.21+9"
fi
export PATH="←!—KATEX_INLINE_0⟶{PATH}"
export PYSPARK_PYTHON="$(pwd)/.venv/bin/python"
export PYSPARK_DRIVER_PYTHON="$(pwd)/.venv/bin/python"

# Confirm Kafka broker availability
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 --list
```

3. Step 1: Dissecting the Streaming Pipeline `python/s10_stream.py`

Inspect `python/s10_stream.py`. It implements the canonical Structured Streaming pattern:

1. **Kafka Stream Read:**
   ```
   python
   raw_stream = (
       spark.readStream
       .format("kafka")
       .option("kafka.bootstrap.servers", "localhost:9092")
       .option("subscribe", "orders.events.v1")
       .option("startingOffsets", "latest")
       .option("failOnDataLoss", "false")
       .load()
   )
   ```
2. **Schema Enforcement & Watermark:**
   ```
   python
   parsed_stream = (
       raw_stream
       .select(F.from_json(F.col("value").cast("string"), order_schema).alias("data"))
       .select("data.*")
       .withWatermark("timestamp", "10 minutes")
   )
   ```
3. **Windowed Aggregation:**
   ```
   python
   windowed_revenue = (
       parsed_stream
       .groupBy(
           F.window(F.col("timestamp"), "10 minutes"),
           F.col("store_id")
       )
       .agg(F.sum("amount").alias("total_sales"), F.count("order_id").alias("order_count"))
   )
   ```
4. **Fault-Tolerant Sink Write:**
   ```
   python
   query = (
       windowed_revenue.writeStream
       .format("console")
       .outputMode("update")
   ```

```
        .option("checkpointLocation", "/tmp/spark_checkpoints/s10_orders")
        .trigger(processingTime="5 seconds")
        .start()
    )
```

4. Step 2: Executing the Streaming Query & Ingesting Events

Start the streaming consumer script:

```
python python/s10_stream.py &
STREAM_PID=$!
echo "Spark Structured Streaming running with PID: ${STREAM_PID}"
```

In a separate terminal, publish a batch of events with simulated timestamps:

```
printf '{"order_id": "ORD-101", "store_id": "STR-01", "amount": 250.0, "timestamp": "2026-11
-07T12:00:00Z"}\n{"order_id": "ORD-102", "store_id": "STR-01", "amount": 150.0, "timestamp":
"2026-11-07T12:05:00Z"}\n{"order_id": "ORD-103", "store_id": "STR-02", "amount": 400.0, "tim
estamp": "2026-11-07T12:08:00Z"}\n' | docker exec -i kafka1 /opt/kafka/bin/kafka-console-pro
ducer.sh --bootstrap-server kafka1:9092 --topic orders.events.v1
```

Observe the console output displaying micro-batch aggregations.

To stop the stream gracefully:

```
kill -SIGTERM ${STREAM_PID}
```

5. Step 3: Inspecting the Checkpoint Directory

Explore the physical files created by Spark's write-ahead log in the checkpoint directory:

```
ls -la /tmp/spark_checkpoints/s10_orders/
ls -la /tmp/spark_checkpoints/s10_orders/offsets/
cat /tmp/spark_checkpoints/s10_orders/offsets/0
```

Notice:
- `offsets/` : Contains the exact Kafka starting and ending offsets for each micro-batch.
- `commits/` : Confirms successful micro-batch completion.
- `state/` : Embedded RocksDB or HDFS state delta files storing active window aggregations.

6. Step 4: Automated Testing with Pytest

Run the automated test suite to verify watermarking and window calculations:

```
pytest tests/test_s10_streaming.py -v
```

Expected Output:

```
tests/test_s10_streaming.py::test_watermark_discards_late_events PASSED       [ 50%]
tests/test_s10_streaming.py::test_window_aggregation_computes_exact_sum PASSED [100%]
============================== 2 passed in 5.34s ===============================
```

7. Step 5: Master Verification Script

Run the session verification script:

```
bash Trainer/S10_05_verify_s10.sh
```

Expected Output:

```
[PASS] S10 Spark Structured Streaming & Watermarking verified.
```

🌟 Gold Stretch Challenge: Checkpoint Recovery Under Sudden Crash

> **Distinguished Engineer Challenge:** If you complete the core lab before time expires, simulate a hard executor crash and verify zero data loss:
> 1. Start the streaming pipeline and publish 1,000 synthetic records continuously.
> 2. In the middle of processing, send a hard `kill -9` signal to the Spark driver process.
> 3. Verify that the checkpoint directory `/tmp/spark_checkpoints/s10_orders/` remains intact.
> 4. Restart the streaming application with the exact same checkpoint directory.
> 5. Inspect the logs and verify that Spark automatically reads the latest commit file, resumes reading from the exact Kafka partition offsets where it crashed, and emits accurate aggregates with zero duplicate counts.